

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T
Semester IV) AI(Artificial Intelligence)

Practical VI

Roll No.:T052	Name:Divyanjali yadav
Class:TYIT	Batch:01
Date of Assignment:19-08-2026	Date/Time of Submission:19-08-2026

Probability Reasoning (Bayesian Networks & Inference)

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using Pgmpy.

Code:

```
# Bayesian Network using pgmpy
# Example: Burglary-Alarm Network
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
# Step 2: Define probability of Burglary
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[
        [0.999],
        [0.001]
    ],
    state_names={
        "Burglary": ["False", "True"]
    }
)
# Define probability of Earthquake
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[
        [0.998],
        [0.002]
```

```

    ],
    state_names={
        "Earthquake": ["False", "True"]
    }
)
# P(Alarm | Burglary, Earthquake)
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)
# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],

```

```

state_names={
    "MaryCalls": ["False", "True"],
    "Alarm": ["False", "True"]
}
)
# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)
# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())
# Step 5: Perform inference
inference = VariableElimination(model)
# Query: Probability of Burglary when John and Mary both call
# and MaryCalls=True
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
)
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)

```

Output:

```

Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
DivyanjaliT052

```

Example:**Code:**

```

# Bayesian Network - Medical Diagnosis Example
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Disease", "Symptoms")
])
# Step 2: Define probability of Disease
cpd_disease = TabularCPD(
    variable="Disease",
    variable_card=2,
    values=[
        [0.90], # Disease = False
        [0.10] # Disease = True
    ],

```

```

state_names={
    "Disease": ["False", "True"]
}
)
# Step 3: Define probability of Symptoms
# P(Symptoms | Disease)
cpd_symptoms = TabularCPD(
    variable="Symptoms",
    variable_card=2,
    values=[
        [0.80, 0.20], # Symptoms = False
        [0.20, 0.80] # Symptoms = True
    ],
    evidence=["Disease"],
    evidence_card=[2],
    state_names={
        "Symptoms": ["False", "True"],
        "Disease": ["False", "True"]
    }
)
# Step 4: Add CPDs to the model
model.add_cpds(
    cpd_disease,
    cpd_symptoms
)
# Step 5: Check whether the model is valid
print("Is Bayesian Network valid?", model.check_model())
# Step 6: Perform inference
inference = VariableElimination(model)
# Query 1: Probability of Disease
result1 = inference.query(
    variables=["Disease"]
)
print("\nProbability of Disease:")
print(result1)
# Query 2: Probability of Disease
# when Symptoms = True
result2 = inference.query(
    variables=["Disease"],
    evidence={"Symptoms": "True"}
)
print("\nProbability of Disease given Symptoms=True:")
print(result2)

```

Output:-

```

Is Bayesian Network valid? True

Probability of Disease:
+-----+-----+
| Disease | phi(Disease) |
+-----+-----+
| Disease(False) | 0.9000 |
+-----+-----+
| Disease(True) | 0.1000 |
+-----+-----+

Probability of Disease given Symptoms=True:
+-----+-----+
| Disease | phi(Disease) |
+-----+-----+
| Disease(False) | 0.6923 |
+-----+-----+
| Disease(True) | 0.3077 |
+-----+-----+
Divyanjali T052

```

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code:

```
# Posterior Probability using Bayesian Network
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Step 1: Create Bayesian Network structure
```

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
```

```
# Step 2: Define probability of Burglary
```

```
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[
        [0.999],
        [0.001]
    ],
    state_names={
        "Burglary": ["False", "True"]
    }
)
```

```
# Step 3: Define probability of Earthquake
```

```
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[
        [0.998],
        [0.002]
    ],
    state_names={
        "Earthquake": ["False", "True"]
    }
)
```

```
# Step 4: Define probability of Alarm
```

```
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
```

Output:

```
Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
+-----+-----+
| Burglary | phi(Burglary) |
+-----+-----+
| Burglary(False) | 0.6264 |
+-----+-----+
| Burglary(True) | 0.3736 |
+-----+-----+
Divyanjali T052
```

```

        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
# Step 5: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm
)
# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())
# Step 7: Perform inference
inference = VariableElimination(model)
result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)
print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)

```

Example:-

Code: # Bayesian Network - Weather and Picnic Example

```

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Weather", "Picnic")
])
# Step 2: Define probability of Weather
cpd_weather = TabularCPD(
    variable="Weather",
    variable_card=2,
    values=[
        [0.60], # Weather = Rainy
        [0.40] # Weather = Sunny
    ],
    state_names={
        "Weather": ["Rainy", "Sunny"]
    }
)
# Step 3: Define probability of Picnic
# P(Picnic | Weather)
cpd_picnic = TabularCPD(
    variable="Picnic",

```

```

variable_card=2,
values=[
    [0.90, 0.20], # Picnic = False
    [0.10, 0.80] # Picnic = True
],
evidence=["Weather"],
evidence_card=[2],
state_names={
    "Picnic": ["False", "True"],
    "Weather": ["Rainy", "Sunny"]
}
)
# Step 4: Add CPDs to the model
model.add_cpds(
    cpd_weather,
    cpd_picnic
)
# Step 5: Check whether the model is valid
print("Is Bayesian Network valid?", model.check_model())
# Step 6: Perform inference
inference = VariableElimination(model)
# Step 7: Find posterior probability
# P(Picnic | Weather = Sunny)
result = inference.query(
    variables=["Picnic"],
    evidence={"Weather": "Sunny"}
)
# Step 8: Display result
print("\nPosterior Probability:")
print("P(Picnic | Weather = Sunny)")
print(result)
print("DivyanjaliTo52")

```

Output:

```

Is Bayesian Network valid? True

Posterior Probability:
P(Picnic | Weather = Sunny)
+-----+-----+
| Picnic | phi(Picnic) |
+=====+=====+
| Picnic(False) |      0.2000 |
+-----+-----+
| Picnic(True) |      0.8000 |
+-----+-----+
DivyanjaliTo52

```